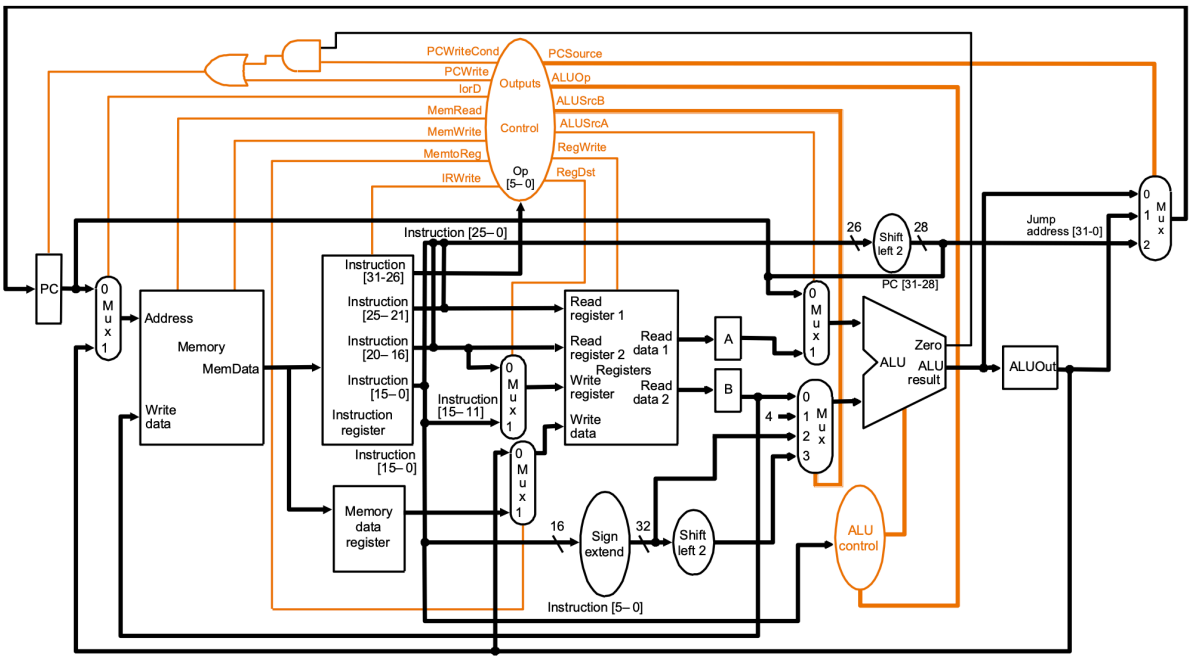
Lab 4 Report

과제 개요

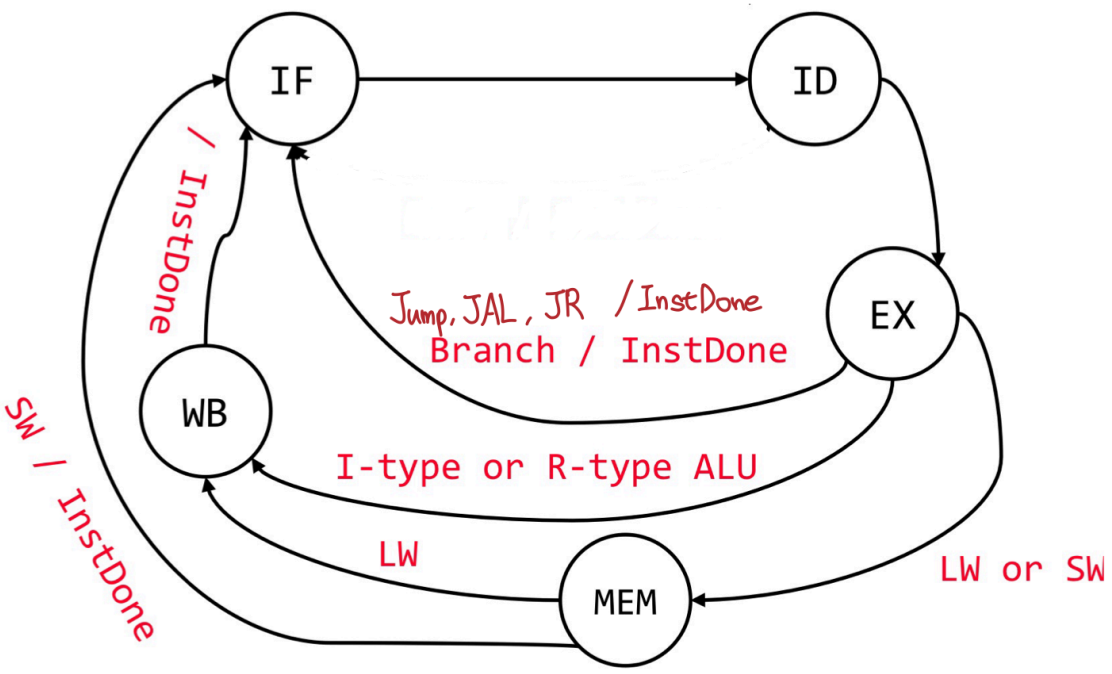
이번 과제는 지난 HW3에서 구현하였던 Single-cycle CPU를 발전시켜 Multi-cycle CPU를 구현하는 과제다.



HW1에서 구현하였던 ALU와 HW2에서 구현하였던 RF를 바탕으로 HW3에서 구현하였던 Single-cycle CPU `CPU.v`와 컨트롤 신호를 위한 `CTRL.v`를 적절하게 수정하여 Multi-cycle CPU를 만드는 것이 목표다. 이 목표를 달성하기 위해서 추가적인 컨트롤 신호가 필요하고, Finite State Machine을 정의하여 명령어 유형 별로 stage 간의 이동을 정의해주어야 한다. 또한 스켈레톤 코드로 제공된 메모리 파일은 명령어와 데이터 포트가 하나로 통일되어 있어, 이를 컨트롤 신호로 구분해주어야 한다.

추가로 Multi-cycle CPU의 장점인 자원 공유를 구현해주어야 한다. 각각 IF와 EX, IF와 MEM stage를 수행하는 ALU와 메모리가 컨트롤 신호를 통해 작업이 구분된다.

JAL/JR 구현을 위한 FSM



기존 FSM을 변형하여 Jump 명령어를 ID 단계가 아닌 EX 단계를 마치고 InstDone 신호를 보내도록 하였고, JAL과 JR 명령어 역시 EX 단계를 마치고 InstDone 신호를 보내어 명령어 실행을 완료한다. 프로그래밍 편의를 위해 IF와 ID 단계에서 수행하는 컨트롤 신호 제어를 통일하고자 하였고, 따라서 J-type 명령어가 EX 단계까지 실행하도록 FSM을 수정하였다.

Jump의 경우 ID 단계에서 바로 명령어를 완료할 수 있으나, JR의 경우 레지스터에서 값을 읽어와야 그 주소로 이동할 수 있고, ID 단계에서 레지스터를 읽자마자 동일한 cycle 내에서 PC를 덮어쓰려고 하면 critical path가 길어질 위험이 있다. 또 JAL의 경우 돌아올 주소(`PC + 4`)를 31번 레지스터에 써야하는데, 만약 ID 단계에서 이를 수행한다면 ID 단계의 본래 목적인 '레지스터 읽기'와 '레지스터 쓰기'가 동시에 충돌할 수 있다.

구현 과정

```
    output reg PCWrite,
    output reg PCWriteCond,
    output reg PCWriteCondNeq,
    output reg IorD,
    output reg MemRead,
    output reg MemWrite,
    output reg IRWrite,
    output reg [1:0] MemtoReg,
    output reg [1:0] RegDst,
    output reg RegWrite,
    output reg ALUSrcA,
    output reg [1:0] ALUSrcB,
    output reg [3:0] ALUOp,
    output reg [1:0] PCSource,
    output reg SignExtend
```

먼저 `CTRL.v`부터 수정하였다. 추가적인 컨트롤 신호로 `PCWrite`, `PCWriteCond`, `PCWriteCondNeq`, IorD, `IRWrite`, `PCSource`가 추가되었고, `ALUSrc`는 각각 `ALUSrcA`와 `ALUSrcB`로 나뉘었다. JAL/JR 명령어 구현을 위해 `RegDst`, `MemtoReg`, `PCSource`를 기존 1-bit에서 2-bit로 확장하여 MUX 경로를 추가하였다.

```
    localparam
        IF  = 3'd0,
        ID  = 3'd1,
        EX  = 3'd2,
        MEM = 3'd3,
        WB  = 3'd4;

    reg [2:0] state, next_state;
```

과제 명세에 명시된 FSM을 구현하기 위해 ID부터 WB까지의 stage를 만들고, `state`와 `next_state`로 현재 상태와 이동할 상태를 나타내었다.

```
41      always @(*) begin
42          case(state)
43              IF: next_state = ID;
44              ID: next_state = EX;
45              EX: begin
46                  case(opcode)
47                      `OP_LW, `OP_SW: next_state = MEM;
48                      `OP_BEQ, `OP_BNE, `OP_J, `OP_JAL: next_state = IF;
49                      `OP_RTYPE: next_state = (funct == `FUNCT_JR) ? IF : WB;
50                      default: next_state = WB;
51                  endcase
52              end
53              MEM: begin
54                  if (opcode == `OP_LW) next_state = WB;
55                  else next_state = IF;
56              end
57              WB: next_state = IF;
58              default: next_state = IF;
59          endcase
60      end
```

IF와 ID는 모든 명령어가 거치는 단계이므로 고정하고, EX 단계에서 명령어 유형에 따라 분기하여 알맞는 다음 상태로 이동하도록 하였다.

```
69          case(state)
70              IF: begin
71                  MemRead = 1; IRWrite = 1; ALUSrcA = 0; ALUSrcB = 2'b01;
72                  PCWrite = 1; PCSource = 2'b00; IorD = 0;
73              end
74
75              ID: begin
76                  ALUSrcA = 0; ALUSrcB = 2'b11;
77                  SignExtend = 1;
78              end
```

```
80              EX: begin
81                  ALUSrcA = 1;
82                  case(opcode)
83                      `OP_RTYPE: begin
84                          if (funct == `FUNCT_JR) begin
85                              PCWrite = 1; PCSource = 2'b11;
86                          end else begin
87                              ALUSrcB = 2'b00;
88                              case(funct)
89                                  `FUNCT_ADDU: ALUOp = `ALU_ADDU;
90                                  `FUNCT_SUBU: ALUOp = `ALU_SUBU;
91                                  `FUNCT_AND:  ALUOp = `ALU_AND;
92                                  `FUNCT_OR:   ALUOp = `ALU_OR;
93                                  `FUNCT_XOR:  ALUOp = `ALU_XOR;
94                                  `FUNCT_NOR:  ALUOp = `ALU_NOR;
95                                  `FUNCT_SLT:  ALUOp = `ALU_SLT;
96                                  `FUNCT_SLTU: ALUOp = `ALU_SLTU;
97                                  `FUNCT_SLL:  ALUOp = `ALU_SLL;
98                                  `FUNCT_SRL:  ALUOp = `ALU_SRL;
99                                  `FUNCT_SRA:  ALUOp = `ALU_SRA;
100                                 default:     ALUOp = `ALU_ADDU;
101                             endcase
102                         end
103                 end
104                 `OP_LW, `OP_SW: begin
105                     ALUSrcB = 2'b10; ALUOp = `ALU_ADDU; SignExtend = 1;
106                 end
107                 `OP_BEQ: begin
108                     ALUSrcB = 2'b00; ALUOp = `ALU_SUBU; SignExtend = 1;
109                     PCWriteCond = 1; PCSource = 2'b01;
110                 end
111                 `OP_BNE: begin
112                     ALUSrcB = 2'b00; ALUOp = `ALU_SUBU; SignExtend = 1;
113                     PCWriteCondNeq = 1; PCSource = 2'b01;
```

```
115                 `OP_J: begin
116                     PCWrite = 1; PCSource = 2'b10;
117                 end
118                 `OP_JAL: begin
119                     PCWrite = 1; PCSource = 2'b10;
120                     RegDst = 2'b10; MemtoReg = 2'b10; RegWrite = 1;
121                 end
122                 `OP_ADDIU: begin ALUSrcB = 2'b10; ALUOp = `ALU_ADDU; SignExtend = 1; end
123                 `OP_ANDI:  begin ALUSrcB = 2'b10; ALUOp = `ALU_AND; SignExtend = 0; end
124                 `OP_ORI:   begin ALUSrcB = 2'b10; ALUOp = `ALU_OR; SignExtend = 0; end
125                 `OP_XORI:  begin ALUSrcB = 2'b10; ALUOp = `ALU_XOR; SignExtend = 0; end
126                 `OP_LUI:   begin ALUSrcB = 2'b10; ALUOp = `ALU_LUI; SignExtend = 0; end
127                 `OP_SLTI:  begin ALUSrcB = 2'b10; ALUOp = `ALU_SLT; SignExtend = 1; end
128                 `OP_SLTIU: begin ALUSrcB = 2'b10; ALUOp = `ALU_SLTU; SignExtend = 1; end
129                 default:   begin ALUSrcA = 0; ALUSrcB = 2'b00; end
```

```
133             MEM: begin
134                 if (opcode == `OP_LW) begin
135                     IorD = 1; MemRead = 1;
136                 end else if (opcode == `OP_SW) begin
137                     IorD = 1; MemWrite = 1;
138                 end
139             end
140
141             WB: begin
142                 RegWrite = 1;
143                 case(opcode)
144                     `OP_RTYPE: begin
145                         RegDst = 2'b01; MemtoReg = 2'b00;
146                     end
147                     `OP_LW: begin
148                         RegDst = 2'b00; MemtoReg = 2'b01;
149                     end
150                     default: begin
151                         RegDst = 2'b00; MemtoReg = 2'b00;
152                     end
153                 endcase
154             end
```

각 stage에서 명령어 유형에 맞는 컨트롤 신호를 정의해주었다. JAL 명령어는 `PCSource = 2'b10`으로 설정하여 Instruction Register에서 가져온 Jump target address로 이동시켜 준다. 이 PC latching이 발생하기 전에 기존의 `PC + 4` 값을 `RegWrite`를 1로 설정함으로써 31번 레지스터에 안전하게 저장해둔다. JR 명령어는 `PCWrite`를 1로 설정하여 PC latching을 허용해주고, nextPC로 이전에 JAL 명령어가 31번 레지스터에 저장해둔 `PC + 4` 값을 가져오기 위해 `PCSource`로 `A`를 선택하는 `2'b11`로 설정하였다.

```
43      assign mem_addr = IorD ? ALUOut : PC;
44      assign mem_write_data = B;
45
46      wire [4:0] wr_addr = (RegDst == 2'b10) ? 5'd31 :
47                           (RegDst == 2'b01) ? rd : rt;
48
49      wire [31:0] wr_data = (MemtoReg == 2'b10) ? PC :
50                            (MemtoReg == 2'b01) ? MDR : ALUOut;
51
52      assign operand1 = ALUSrcA ? A : PC;
53      assign operand2 = (ALUSrcB == 2'b11) ? (ext_imm << 2) :
54                        (ALUSrcB == 2'b10) ? ext_imm :
55                        (ALUSrcB == 2'b01) ? 32'd4 : B;
56
57      assign PC_next = (PCSource == 2'b11) ? A :
58                       (PCSource == 2'b10) ? {PC[31:28], immj, 2'b00} :
59                       (PCSource == 2'b01) ? ALUOut : alu_result;
```

다음으로 `CPU.v` 파일을 수정하였다.

- 메모리 자원을 공유하기 위해 `IorD` 컨트롤 신호를 사용하여 1이면 `ALUOut`, 0이면 `PC`를 `mem_addr` 메모리 주소값으로 사용한다.
- 기존의 Single-cycle CPU에서 `RegDst`는 RF에 데이터를 쓸 때, `rt`에 쓰면 0, `rd`에 쓰면 1로 설정하였다. 이제 JAL 명령어가 추가되어 `RegDst`가 2면 31번 레지스터에 데이터를 쓸 수 있도록 하였다.
- 기존의 Single-cycle CPU에서 `MemtoReg`는 RF에 쓰는 데이터의 출처에 따라 ALU의 연산값을 쓰면 0, 메모리에서 가져온 데이터를 쓰면 1로 설정하였다. 이제 JAL 명령어가 추가되어 `MemtoReg`가 2면 IF 단계에서 이미 `PC`가 `PC + 4`로 업데이트되었기 때문에 `PC`를 불러올 수 있도록 하였다.
- 첫번째 피연산자 `operand1`의 경우 `PC + 4`를 계산하기 위한 ALU일 경우 0, 입력 받은 `A`를 사용한 연산을 하기 위한 ALU일 경우 1로 설정하였다.
- 두번째 피연산자 operand2의 경우:
  - 0: 입력받은 `B` 사용
  - 1: `PC + 4`를 계산하기 위해 `4`를 사용
  - 2: I-type 명령어 계산을 위해 sign-extend한 immediate 값 `ext_imm` 사용
  - 3: Branch 명령어의 target address 계산을 위한 `ext_imm << 2` 값 사용
- nextPC는 `PCSource` 신호로 제어해준다. 0이면 ALU의 연산 결과, 1이면 `ALUOut` 값, 2면 jump target address, 3이면 JR 명령어일 때 앞서 JAL 명령어에서 31번 레지스터에 저장해두었던 주소로 이동시켜주기 위해 `A`로 설정하였다.

```
61      wire Zero = (alu_result == 32'd0);
62      wire PCEn = PCWrite | (PCWriteCond & Zero) | (PCWriteCondNeq & ~Zero);
63
64      always @(posedge clk) begin
65          if (rst) begin
66              PC <= 0;
67              IR <= 32'hFFFFFFFF;
68              MDR <= 0; A <= 0; B <= 0; ALUOut <= 0;
69          end else begin
70              if (PCEn) PC <= PC_next;
71              if (IRWrite) IR <= mem_read_data;
72              MDR <= mem_read_data;
73              A <= rd_data1;
74              B <= rd_data2;
75              ALUOut <= alu_result;
76          end
77      end
```

`PCEn`에 PC latching이 허용되는 조건을 위와 같이 설정해주었다. 이 조건을 만족할 때에만 `PC`를 `PC_next`로 바꾸어주고(PC latching enabled), 마찬가지로 IR latching의 조건인 `IRWrite`를 만족할 때에만 instruction register에 값을 읽어올 수 있도록 하였다.

모든 모듈을 instantiate하고 모듈을 연결하여 Multi-cycle CPU를 완성하였다.

테스트

지난 HW3와 동일한 테스트케이스와 테스트벤치를 사용하기 때문에 테스트케이스를 동시에 실행하기 위한 `run_all.sh` 파일을 재사용하였다.

```
yjkim0306 ~/Documents/3-1/컴퓨터구조론/lab/HW04 Skeleton Code  main bash run_all.sh
Compiling Verilog files...
========================================
          Running Testcase 1
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 24895000 (1ps)
========================================
          Running Testcase 2
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 7505000 (1ps)
========================================
          Running Testcase 3
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 5805000 (1ps)
========================================
          Running Testcase 4
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 28205000 (1ps)
========================================
          Running Testcase 5
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 12625000 (1ps)
========================================
          Running Testcase 6
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 8225000 (1ps)
========================================
          Running Testcase 7
========================================
Loading instruction memory...
Program Terminate

Simulation success!!!
CPU_tb.v:57: $finish called at 20335000 (1ps)
========================================
          All testcases finished!
========================================
```

7개의 테스트케이스 모두 성공하였다.